EUDAMED Update Monitoring

How `eudamed2firstbase` detects, tracks, and reacts to changes in EUDAMED Basic UDI-DI and UDI-DI records.

Document version: 2026-04-17 · Tool version: see `Cargo.toml` / window title · Source of truth: `src/version_db.rs`, `src/download.rs`

1. Why tracking matters

EUDAMED is a moving database: manufacturers update addresses, status changes from `ON_MARKET` to `NO_LONGER_ON_THE_MARKET`, Notified Bodies link new certificates, package hierarchies are extended. Re-downloading and re-pushing 100k+ devices on every run is wasteful and triggers GS1 re-publication errors (e.g. `SYS25` "must be later than previous", `097.029/036` updateable-rule blocks). Instead, the tool keeps a per-device version journal and only acts on what has actually changed.

2. The two API records we track

Record	EUDAMED endpoint	What it carries
UDI-DI (Detail)	`GET /api/devices/udiDiData/{uuid}?languageIso2Code=en`	Per-package device data: GTIN/primary DI, trade name, clinical sizes, market info, substances, warnings, packaging hierarchy, device status, product designer, related devices.
Basic UDI-DI (BUDI)	`GET /api/devices/basicUdiData/udiDiData/{uuid}?languageIso2Code=en`	Per-product-family record: MDR/IVDR mandatory booleans (active, implantable, measuringFunction, multiComponent, tissue), risk class, regulatory act (MDR/IVDR/MDD/AIMDD/IVDD), manufacturer + Authorised Representative SRN, certificates (DeviceCertificateInfo and CertificateLink).
Listing	`GET /api/devices/udiDiData?page=N&pageSize=300&srn=<SRN>`	Paginated index used as the discovery layer. Carries `versionNumber` (UDI) and `basicUdiDataVersionNumber` (BUDI) for every UUID — this is the cheap signal that drives the pre-download check.

> Every section in both Detail and BUDI records carries its own `versionNumber` and `versionDate` (or `lastUpdateDate` for contacts). EUDAMED versions sections independently — a manufacturer address change bumps `manufacturer.versionNumber` without touching `udiDi.versionNumber`.

3. What gets tracked, per UUID

One row in the SQLite table `udi_versions` per UDI-DI UUID. Nine independent sections plus a fast-path content hash:

Section	Source	Version field(s)	DB columns
UDI root	Detail JSON	`versionNumber`, `versionDate`	`udi_version`, `udi_date`
Basic UDI-DI	BUDI JSON	`versionNumber`, `versionDate`	`budi_version`, `budi_date`
Manufacturer	BUDI → `manufacturer`	`versionNumber`, `lastUpdateDate`	`mfr_version`, `mfr_date`
Authorised Rep	BUDI → `authorisedRepresentative`	`versionNumber`, `lastUpdateDate`	`ar_version`, `ar_date`
Certificates	BUDI → `deviceCertificateInfoListForDisplay[]`	`versionNumber` per cert	`cert_versions` (JSON array)
Package	Detail → `containedItem`	`versionNumber`, `versionDate`	`pkg_version`, `pkg_date`
Market info	Detail → `marketInfoLink`	`versionNumber`, `versionDate`	`market_version`, `market_date`
Device status	Detail → `deviceStatus`	status code, `statusDate`	`device_status`, `status_date`
Product designer	Detail → `productDesigner`	`versionNumber`, `versionDate`	`designer_version`, `designer_date`
Detail hash	Full Detail JSON body	SHA-256	`detail_hash` (fast-path)
Last sync	Tool clock	ISO-8601 timestamp	`last_synced`

4. Storage layout

SQLite, WAL mode, located at `$APP_DATA/db/version_tracking.db` (`~/eudamed2firstbase/db/` on Linux/macOS, `%USERPROFILE%\eudamed2firstbase\db\` on Windows, the sandbox container under the App Store build). Three operational tables share the file:

- `udi_versions` — the per-UUID version journal described above.
- `listing_cache` — paginated SRN listing snapshot (UUID, SRN, manufacturer name, primary DI, trade name, risk class, device status, `versionNumber`, `basicUdiDataVersionNumber`, `listed_at`). Drives the cheap pre-download check.
- `push_log` + `request_log` + `swissdamed_push_log` — outbound push history (separate document).

```
CREATE TABLE udi_versions (
    uuid              TEXT PRIMARY KEY,
    gtin              TEXT NOT NULL DEFAULT '',
    detail_hash       TEXT NOT NULL DEFAULT '',
    udi_version       INTEGER, udi_date      TEXT,
    budi_version      INTEGER, budi_date     TEXT,
    mfr_version       INTEGER, mfr_date      TEXT,
    ar_version        INTEGER, ar_date       TEXT,
    cert_versions     TEXT,                              -- JSON array, e.g. "[1,2,1]"
    pkg_version       INTEGER, pkg_date      TEXT,
    market_version    INTEGER, market_date   TEXT,
    device_status     TEXT,    status_date   TEXT,
    designer_version  INTEGER, designer_date TEXT,
    last_synced       TEXT NOT NULL DEFAULT ''
);
CREATE INDEX idx_gtin        ON udi_versions(gtin);
CREATE INDEX idx_last_synced ON udi_versions(last_synced);
```

5. The detection algorithm

1. **Discovery (cheap).** For each manufacturer SRN, fetch the listing pages in parallel (default 10 threads). Each page row gives us `(uuid, versionNumber, basicUdiDataVersionNumber)`. Rows are upserted into `listing_cache`.

2. **Pre-download filter (`filter_unchanged`).** For every listing row, look up `udi_versions[uuid]`:
   - If both `db.udi_version == listing.versionNumber` *and* `db.budi_version == listing.basicUdiDataVersionNumber`, mark **unchanged** — skip detail+BUDI fetch entirely.
   - Otherwise add the UUID to `need_download`.

   Special case: if the DB has no BUDI version yet but the listing carries one, the device is re-downloaded so we can capture the new BUDI data.

3. **Parallel detail + BUDI fetch.** For every UUID in `need_download`, fetch `/{uuid}` → `eudamed_json/detail/<uuid>.json` and `/basicUdiData/udiDiData/{uuid}` → `eudamed_json/basic/<uuid>.json`. Three retries with exponential backoff; per-10-file progress counters; full log to `eudamed_json/log/download.log`.

4. **Indexing (`index_detail_versions`).** Newly downloaded detail files are parsed in parallel via Rayon, run through `extract_detail_versions()`, and upserted into `udi_versions` in a single transaction. The same path runs for the initial bulk import (`index_all_detail_versions`) and subsequent incremental syncs.

5. **BUDI version backfill.** The `basicUdiDataVersionNumber` from the listing is written back to `udi_versions.budi_version` in a transaction so the next run can compare both UDI and BUDI versions without having to refetch BUDI files.

6. **Conversion / change detection (`detect_changes`).** When the conversion pipeline runs (`cargo run firstbase` or the GUI's one-click flow), each detail file is hashed:
   - **Fast path:** if `SHA-256(file) == db.detail_hash`, return `UNCHANGED` immediately — no JSON re-parse, no re-conversion.
   - **Slow path:** compare each tracked version field independently and produce a `ChangeSet`: `{ udi_changed, budi_changed, mfr_changed, ar_changed, cert_changed, pkg_changed, market_changed, status_changed, designer_changed }`.

The summary string — e.g. `NEW`, `MFR+CERT`, `STATUS+MARKET`, `UNCHANGED` — is emitted to the log so operators can see at a glance what changed for each UUID.

6. Why two layers (hash + per-section)

Layer	Cost	What it answers
SHA-256 of full Detail JSON	~µs per file	"Is anything different at all?"
Per-section version compare	~tens of µs	"*Which* sections changed?"
Listing version pre-check	One HTTP page per 300 UUIDs	"Should we even fetch this UUID?"

The three layers compose: the listing check avoids HTTP entirely for unchanged devices, the hash short-circuits parse on hits, and the per-section compare narrows GS1 re-pushes to the attributes that actually moved.

7. The `check` command (operational entry point)

One command runs the full incremental loop — intended for cron / CI:

```
cargo run check srns.txt [--threads N]
```

1. Read SRNs from the file (one per line, comments allowed).
2. Run discovery + pre-download filter (steps 1–2).
3. If `need_download` is empty: print "All N devices unchanged" and exit 0 with no further work.
4. Otherwise: download (3), index (4), backfill (5), convert only the changed UUIDs (6), and push to GS1 firstbase (Live/CreateMany → AddMany with polling).

8. Inspecting the journal

```
sqlite3 ~/eudamed2firstbase/db/version_tracking.db
> SELECT COUNT(*) FROM udi_versions;
> SELECT uuid, gtin, udi_version, budi_version, device_status, last_synced
  FROM udi_versions
  ORDER BY last_synced DESC LIMIT 10;
> SELECT device_status, COUNT(*) FROM udi_versions
  GROUP BY device_status;
> SELECT uuid, mfr_version, mfr_date FROM udi_versions
  WHERE mfr_date > '2026-01-01';
```

9. Failure modes & their telemetry

Failure	Symptom	How we recover
HTTP timeout / 5xx during detail fetch	UUID stays in `need_download`, file missing on disk	Up to 3 retries with exponential backoff, then a 5-retry completeness sweep at the end of the run; remaining gaps are logged.
EUDAMED returns malformed JSON	Parse fails in `extract_detail_versions`	Default-initialised `VersionRecord` kept; `detail_hash` still stored, so file is re-tried next run if it changes.
BUDI cache miss	`budi_version` NULL	Listing-driven backfill on next run; converter falls back to `false` for MDR booleans (logged with reduced confidence).
Section version regresses (n-1)	Treated as a change	The compare uses `!=`, not `<`, so EUDAMED rollbacks re-trigger conversion and push.

10. End-to-end timeline

```
+-------------------+    +-------------------+    +-----------------+    +-----------+
| Listing fetch     | -> | filter_unchanged  | -> | parallel detail | -> | index +   |
| (SRN, page=N)     |    | UDI? BUDI?        |    | + BUDI download |    | upsert    |
| listing_cache     |    | DB lookup         |    | + log           |    | versions  |
+-------------------+    +-------------------+    +-----------------+    +-----------+
                               |                                                 |
                               | unchanged                                       v
                               v                                        +-------------+
                         skip download                                  | detect_     |
                                                                        | changes()   |
                                                                        | per UUID    |
                                                                        +-------------+
                                                                              |
                                                                              v
                                                                        +-------------+
                                                                        | convert &   |
                                                                        | push only   |
                                                                        | changed     |
                                                                        +-------------+
```

11. Files of interest

- `src/version_db.rs` — schema, `VersionRecord`, `ChangeSet`, `extract_detail_versions`, `merge_budi_versions`, `detect_changes`.
- `src/download.rs` — `filter_unchanged`, `index_detail_versions`, `index_all_detail_versions`, `listing_cache` schema, BUDI backfill transaction.
- `src/main.rs` — `check` subcommand wiring.
- `db/version_tracking.db` — the live journal.
- `eudamed_json/log/download.log` — per-UUID download trace.

Zeno Davatz
ywesee GmbH · ywesee.com